

Linux Troubleshooting for DevOps Interviews

A practical, scenario-driven training plan for Vartha

Built around your AWS EC2 free tier lab. Designed for interview readiness, not command memorization.

The Core Principle

Interviewers don't test commands. They test **mental model**, **speed to signal**, and **communication**. Memorizing 100 commands won't help if you don't know which to run first. Drill the mental model below until it is reflex.

The 7 Buckets

Every Linux problem falls into one of these: **CPU, Memory, Disk, Network, Process, Configuration, Logs**. Your job is to identify the bucket in 30 seconds, then drill down.

The 4 Vital Signs (run these first, always)

Bucket	First Command	What You're Looking For
CPU	uptime / top	Load average, top processes
Memory	free -h	Used vs available, swap usage
Disk	df -h / df -i	Space and inode exhaustion
Network	ss -tulpn	Listening ports, established conns

Drill: When asked any scenario, say out loud: "First, I'd check the four vital signs — load, memory, disk, network — to identify which bucket the problem is in." This single sentence signals senior thinking.

Lab Setup (Do This First)

Launch 2 EC2 instances in the same VPC.

Instance A — 'victim': Ubuntu 22.04 LTS, t2.micro. This is where you break things.

Instance B — 'client': Ubuntu 22.04 LTS, t2.micro. This is where you test connectivity from.

Security group: Allow SSH (22), HTTP (80), HTTPS (443), ICMP from your IP and from each other.

Install on the victim:

```
sudo apt update
sudo apt install -y nginx stress-ng htop net-tools tcpdump iotop sysstat dnsutils
curl jq
```

Cost discipline

Free tier covers 750 hours/month for ONE t2.micro. Two instances running 24x7 will exceed it. **Stop instances when not practicing.** Set an AWS billing alert at \$1. This is non-negotiable.

Week 1 — Foundations and the Four Vital Signs

Goal: be able to run the 4 vital signs in 30 seconds, and explain what each output means.

Day 1-2: CPU and Load

```
uptime
top # press 1 to see per-CPU, P to sort by CPU
htop
mpstat -P ALL 2 5 # CPU per core, 5 samples 2s apart
pidstat -u 2 5 # CPU per process
```

Must understand: Load average is the number of processes runnable or waiting on I/O, averaged over 1/5/15 minutes. A load of 4 on a 2-core machine = oversubscribed. A load of 4 on an 8-core machine = fine. Always check core count: nproc.

Day 3-4: Memory and Swap

```
free -h
vmstat 2 5
cat /proc/meminfo | head -20
ps aux --sort=-%mem | head -10
```

Trap: 'available' is what matters, not 'free'. Linux uses free RAM as cache; high cache usage with low 'free' is normal. Swap activity (si/so in vmstat) is the real red flag.

Day 5: Disk and I/O

```
df -h # filesystem space
df -i # inode usage (the silent killer)
du -sh /* 2>/dev/null | sort -h # what's eating disk
iostat -xz 2 5 # per-device I/O stats
iotop # processes doing I/O
```

Interview gem: 'Disk is full' but df -h shows space free → check df -i for inode exhaustion (millions of tiny files). Few candidates know this. Mention it and you stand out.

Day 6-7: Network basics

```
ss -tulpn # listening ports + processes (replaces netstat)
ss -s # socket summary
ip addr # interfaces (replaces ifconfig)
ip route # routing table
ping, traceroute, dig, curl -v
```

Week 2 — Bucket-by-Bucket Drills

Goal: for each bucket, you can break it, diagnose it, and fix it on your victim EC2.

Drill 1 — CPU saturation

Symptom: Server is slow, users complain of lag.

Confirm bucket: uptime

Drill commands:

```
top -o %CPU
pidstat -u 2 5
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head
```

Break it (on victim):

```
stress-ng --cpu 2 --timeout 300s &
```

Fix it:

```
kill <pid> # or pkill stress-ng
```

Interview question this prepares you for: *Server load average is 8 on a 2-core machine. Walk me through your debug.*

Drill 2 — Memory leak / OOM

Symptom: App keeps getting killed; nothing in app logs.

Confirm bucket: free -h ; dmesg | grep -i 'killed process'

Drill commands:

```
ps aux --sort=-%rss | head
cat /proc/<pid>/status | grep -i vm
journalctl -k | grep -i oom
```

Break it (on victim):

```
stress-ng --vm 1 --vm-bytes 800M --timeout 120s
```

Fix it:

```
Identify the process; fix the leak or add memory; tune oom_score_adj if it's a
critical service.
```

Interview question this prepares you for: *App is being killed every few hours, no error in app logs. Where do you look?*

Drill 3 — Disk full

Symptom: Application is failing with 'No space left on device'.

Confirm bucket: `df -h ; df -i`

Drill commands:

```
du -sh /var/log/* | sort -h
find / -type f -size +500M 2>/dev/null
lsof | grep deleted # files held open after deletion
```

Break it (on victim):

```
fallocate -l 7G /tmp/bigfile
```

Fix it:

```
rm /tmp/bigfile # and rotate/truncate big logs; restart processes holding deleted files
```

Interview question this prepares you for: *Disk shows 100% but du shows much less used. What's happening?*

Senior answer: A process likely deleted a large file but still has it open. The space isn't released until the process closes the fd or restarts. Use `lsof | grep deleted`.

Drill 4 — Port not reachable

Symptom: Service is running but clients can't connect.

Confirm bucket: `ss -tulpn | grep <port>`

Drill commands:

```
curl -v localhost:<port>
curl -v <public-ip>:<port> from client EC2
sudo iptables -L -n
Check AWS security group
```

Break it (on victim):

```
sudo ufw enable; sudo ufw deny 80 # or remove SG rule for port 80
```

Fix it:

```
sudo ufw allow 80 # or re-add SG rule
```

Interview question this prepares you for: *Nginx is running on port 80 but external clients get 'connection refused'. Debug it.*

Mental order for network issues: Is the process listening? On which interface (0.0.0.0 vs 127.0.0.1)? Is the local firewall blocking? Is the cloud security group blocking? Is DNS resolving? Is the route correct? Walk this ladder, never skip steps.

Drill 5 — DNS issue

Symptom: App can't reach an external API; getting 'Name or service not known'.

Confirm bucket: dig <hostname> ; cat /etc/resolv.conf

Drill commands:

```
nslookup <hostname>
dig +trace <hostname>
systemd-resolve --status
```

Break it (on victim):

```
sudo sh -c 'echo nameserver 1.1.1.99 > /etc/resolv.conf'
```

Fix it:

```
sudo sh -c 'echo nameserver 8.8.8.8 > /etc/resolv.conf'
```

Interview question this prepares you for: *Some apps work, others can't resolve external hostnames. What do you check?*

Drill 6 — Service won't start

Symptom: systemctl start nginx fails.

Confirm bucket: systemctl status nginx

Drill commands:

```
journalctl -u nginx -n 50 --no-pager
nginx -t # config test
ss -tulpn | grep :80 # port already used?
```

Break it (on victim):

```
sudo sed -i 's/listen 80;/listen 80 invalid_syntax;/'
/etc/nginx/sites-enabled/default
```

Fix it:

```
Revert the change; nginx -t to verify; systemctl start nginx
```

Interview question this prepares you for: *A service refuses to start after a config change. Walk me through your debug.*

Drill 7 — File permission / access

Symptom: App logs 'permission denied' opening a file.

Confirm bucket: `ls -l <file> ; id <user>`

Drill commands:

```
namei -l <path> # shows perms on every dir in the path
getfacl <file> # ACLs
ps -eo user,pid,cmd | grep <app>
```

Break it (on victim):

```
sudo chmod 600 /var/log/syslog && sudo chown root:root /var/log/syslog
```

Fix it:

```
Restore correct ownership/perms; never blanket 777.
```

Interview question this prepares you for: *App can't write to its log file. How do you diagnose without breaking other things?*

Week 3 — Full Scenario Drills (Interview Format)

Goal: practice answering scenarios **out loud**. Time yourself: 5 minutes per scenario. Record yourself if possible. The communication is half the interview.

Scenario A — The 502

Question: Your team's web app behind nginx is returning 502 Bad Gateway intermittently. Logs show the upstream is timing out. Walk me through your debugging.

How to answer: Expected flow: 1) Is upstream process alive? (systemctl/ps) 2) Is upstream port reachable from nginx host? (curl localhost:upstream_port) 3) Resource pressure on upstream? (top, free, iostat) 4) Connection limits / file descriptor exhaustion? (ulimit -n, ss -s) 5) Slow queries downstream? (DB / external API latency). Mention nginx error log location: /var/log/nginx/error.log.

Scenario B — The mystery slowdown

Question: Application response time doubled this morning. No deploys overnight. CPU and memory look normal. Debug.

How to answer: Expected flow: 1) Define 'normal' - check historical metrics. 2) Network latency to dependencies (DB, cache, external APIs). 3) Disk I/O wait (iostat - look at %iowait and await). 4) Did data volume grow? (DB row counts, queue depths). 5) Was there a config change / cert rotation / DNS change? Senior signal: ask 'what changed' before running any command.

Scenario C — The full disk that isn't

Question: Monitoring says /var is 100% full. df -h confirms. You delete a 5GB log file. df still shows 100%. Why?

How to answer: Answer: a process still has the file open (held file descriptor). Use lsof | grep deleted to find it. Either restart the process or truncate via /proc/PID/fd/N. This is a classic question and 70% of juniors fail it.

Scenario D — The connection refused

Question: You deployed a new service on port 8080. The service is running. From your laptop you get 'connection refused'. From the EC2 itself, curl localhost:8080 works. Why?

How to answer: Possible causes in order: 1) Service bound to 127.0.0.1 not 0.0.0.0 (check ss -tulpn). 2) Local firewall (ufw/iptables). 3) AWS security group missing inbound rule for 8080. 4) NACL on subnet. 5) Wrong public IP / instance behind NAT. Always walk outward from the process to the client.

Scenario E — The CrashLoopBackOff

Question: A Kubernetes pod is in CrashLoopBackOff. What do you check on the node?

How to answer: On node: kubectl describe pod (events at bottom), kubectl logs --previous, node resources (df, free, top), check kubelet logs (journalctl -u kubelet), image pull errors, readiness/liveness probe failures, OOMKilled events. Bridge from Linux to Kubernetes - shows you understand both layers.

Week 4 — Mock Interview Mode

Goal: rehearse under pressure. Use Claude as your interviewer. Use a real recruiter mock if possible.

Daily routine for week 4

1. Pick 3 random scenarios from week 3 list.
2. Set a 5-minute timer per scenario.
3. Speak your answer OUT LOUD. Record it.
4. Play it back. Note: did you start with vital signs? Did you ask 'what changed'? Did you walk a clear ladder or jump around?
5. Run the actual commands on your EC2 victim to verify your mental model matches reality.

Communication patterns that signal seniority

- Start by clarifying: 'Before I dive in, has anything changed recently — deploys, config, traffic patterns?'
- State your mental model: 'I'd narrow this to one of the four buckets — CPU, memory, disk, network — by running...'
- Separate recovery from root cause: 'First I'd restore service by X. Then I'd find root cause by Y.'
- Acknowledge unknowns: 'If that didn't reveal it, I'd next check Z' — show layered thinking, not memorized answers.
- Mention observability: 'In a real setup I'd correlate this with metrics in Datadog/CloudWatch and traces.'

Common interview traps

- **Jumping to commands.** Always state your hypothesis first.
- **Saying 'I'd Google it'.** Acceptable in life, not in interviews. Have a baseline answer.
- **Memorized flags.** Interviewers don't care about exact flags; they care about the why.
- **Ignoring the cloud layer.** 'Connection refused' could be Linux OR a security group. Mention both.
- **Not asking clarifying questions.** Real engineers ask. Silent diving signals junior.

One-Page Command Cheat Sheet

Need	Command
Load + uptime	uptime
CPU by process	top -o %CPU / pidstat -u 2 5
Memory snapshot	free -h
Memory by process	ps aux --sort=-%rss head
Disk space	df -h
Inode usage	df -i
Disk usage by dir	du -sh /* 2>/dev/null sort -h
Disk I/O per device	iostat -xz 2 5
Disk I/O per process	iotop
Listening ports	ss -tulpn
Socket summary	ss -s
IP / routing	ip addr ; ip route
DNS lookup	dig <host> / nslookup <host>
Connectivity test	curl -v <url> / nc -vz <host> <port>
Service status	systemctl status <svc>
Service logs	journalctl -u <svc> -n 100 --no-pager
Kernel logs	dmesg -T tail -50
OOM kills	dmesg grep -i 'killed process'
Find big files	find / -type f -size +500M 2>/dev/null
Files held after delete	lsof grep deleted
File perms full path	namei -l <path>
Process tree	pstree -p / ps -ef --forest
Open files by process	lsof -p <pid>
Network trace	tcpdump -i any port 80 -nn -c 100

Final thought

Do not try to memorize this document. Do the lab. Break things. Fix them. Talk through your process out loud. Interview readiness comes from **repetition under simulated pressure**, not from reading. Your goal at the end of 4 weeks: hear any Linux scenario and instantly know which bucket to check, in what order, and how to communicate it cleanly.